Solutions to Chapter 5 Exercises:

Exercise 1: Understanding Deep Learning in Networking

Deep learning eliminates the need for manual feature engineering by automatically extracting hierarchical representations.
It scales well with large datasets and adapts to dynamic network environments.

Exercise 2: Limitations of Traditional Machine Learning

Traditional ML depends on handcrafted features, struggles with high-dimensional data, and lacks adaptability.
Examples include static routing, rule-based congestion control, and weak anomaly detection.

Exercise 3: Designing a Feedforward Neural Network

Steps include selecting input features (RTT, packet loss), defining architecture, choosing activation functions, training with backpropagation, and evaluating performance.

Exercise 4: Deep Learning for Routing (XOR Problem)

The XOR problem is non-linearly separable. Neural networks solve it using hidden layers and nonlinear activations, enabling complex decision boundaries.

Exercise 5: Autoencoders in Networking

Autoencoders compress data into latent representations.
Higher compression reduces accuracy, while lower compression improves fidelity but uses more bandwidth.

Exercise 6: CNNs for Network Data

CNNs can model spatial patterns in signal strength maps or traffic matrices,
leveraging locality and shared weights to reduce complexity.

Exercise 7: Sequence Modelling in Networking

Sequence modelling captures time dependencies in traffic and congestion.
Linear models capture trends; deep models capture nonlinear temporal relationships.

Exercise 8: RNN vs LSTM

RNNs suffer from vanishing gradients. LSTMs overcome this with gating mechanisms, making them better for long-term dependencies like traffic prediction.

Exercise 9: Deployment Challenges

Challenges include latency constraints, scalability, privacy concerns, limited labeled data, and computational constraints.

Exercise 10: Future Networks with Generative AI

LLMs and GANs enable traffic simulation, intelligent orchestration, hybrid learning systems, and predictive optimization in autonomous networks.

Exercise 11: Coding Exercise Solution Outline

Simulated data can be generated via OMNeT++ or Mininet.
Real-world data introduces noise and variability.
Data must be cleaned and normalized.

Example model:
- Input: packet loss, RTT, throughput
- Hidden layers with ReLU
- Output layer with sigmoid

Evaluate using accuracy and compare simulation vs real-world performance.